

Homework 3 - Double Pendulum Simulation

Βέμης Νικόλαος December 8, 2025

1 Modeling and Discretization of the system dynamics

The double pendulum system consists of two masses m_1 and m_2 attached to two rods of lengths l_1 and l_2 respectively. The angles of the pendulums with respect to the vertical are denoted by θ_1 and θ_2 . The robot joints are actuated by torques u_1 and u_2 .

The state vector is defined as $x = [q_1, q_2, \dot{q}_1, \dot{q}_2]^T$, where q_i are the angles and $\dot{q}_i$ are the angular velocities. The control input vector is defined as $u = [u_1, u_2]^T$. The system dynamics $\dot{x} = f(x, u)$ can be described by the following equations:

$$\ddot{\mathbf{q}} = M^{-1}(\mathbf{q})(\mathbf{u} - C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + G(\mathbf{q})) \quad (1)$$

where:

$$\begin{aligned} M(\mathbf{q}) &= \begin{bmatrix} l_1^2 m_1 + l_2^2 m_2 + l_1^2 m_2 + 2l_1 m_2 l_2 \cos(q_2) & l_2^2 m_2 + l_1 m_2 l_2 \cos(q_2) \\ l_2^2 m_2 + l_1 m_2 l_2 \cos(q_2) & l_2^2 m_2 \end{bmatrix} \in \mathbb{R}^{2 \times 2} \\ C(\mathbf{q}, \dot{\mathbf{q}}) &= \begin{bmatrix} -2\dot{q}_2 l_1 m_2 l_2 \sin(q_2) & -\dot{q}_2 l_1 m_2 l_2 \sin(q_2) \\ \dot{q}_1 l_1 m_2 l_2 \sin(q_2) & 0 \end{bmatrix} \in \mathbb{R}^{2 \times 2} \\ G(\mathbf{q}) &= \begin{bmatrix} gl_1 m_1 \sin(q_1) + gl_2 m_2 \sin(q_1) \\ gl_2 m_2 \sin(q_1 + q_2) \end{bmatrix} \in \mathbb{R}^{2 \times 1} \\ \mathbf{q} &= \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} \in \mathbb{R}^{2 \times 1} \\ \dot{\mathbf{q}} &= \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} \in \mathbb{R}^{2 \times 1} \end{aligned}$$

and with $g = 9.81 \text{ m/s}^2$, $m_1 = m_2 = 1 \text{ kg}$, $l_1 = l_2 = 0.5 \text{ m}$.

For the integration of the system dynamics, we use the Runge-Kutta 4th order (RK4) method with a time step of $\Delta t = 0.05 \text{ s}$. The discrete-time state update equation is given by:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \quad (2)$$

where:

$$\begin{aligned} \mathbf{k}_1 &= f(\mathbf{x}_k, \mathbf{u}_k) \\ \mathbf{k}_2 &= f\left(\mathbf{x}_k + \frac{\Delta t}{2}\mathbf{k}_1, \mathbf{u}_k\right) \\ \mathbf{k}_3 &= f\left(\mathbf{x}_k + \frac{\Delta t}{2}\mathbf{k}_2, \mathbf{u}_k\right) \\ \mathbf{k}_4 &= f(\mathbf{x}_k + \Delta t\mathbf{k}_3, \mathbf{u}_k) \end{aligned}$$

and $f(x, u)$ represents the continuous-time dynamics of the system.

2 Optimal Trajectory Generation

In order to generate an optimal trajectory for the double pendulum system, we formulate a trajectory optimization problem. The decision vector is defined as:

$$\mathbf{z} = [\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_N, \mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{u}_N]^T \in \mathbb{R}^{(N_x + N_u) \times (N+1)}$$

Where $N_x = 4$ is the dimension of the state vector, $N_u = 2$ is the dimension of the control input vector, and $N = 100$ is the number of time steps. This vector includes the control input for the final state $\mathbf{u}_N$ to satisfy the trapezoidal integration constraints, though it is not used in the running cost.

The optimization aims to minimize the following cost function:

$$J = \sum_{k=0}^{N-1} ((\mathbf{x}_k - \mathbf{x}_{goal})^T Q (\mathbf{x}_k - \mathbf{x}_{goal}) + \mathbf{u}_k^T R \mathbf{u}_k) + (\mathbf{x}_N - \mathbf{x}_{goal})^T Q_f (\mathbf{x}_N - \mathbf{x}_{goal})$$

We substitute $\mathbf{e}_k = \mathbf{x}_k - \mathbf{x}_{goal}$ to simplify the notation, yielding:

$$J = \sum_{k=0}^{N-1} (\mathbf{e}_k^T Q \mathbf{e}_k + \mathbf{u}_k^T R \mathbf{u}_k) + \mathbf{e}_N^T Q_f \mathbf{e}_N \quad (3)$$

where Q , R , and Q_f are weighting matrices that penalize deviations from the goal state and control effort, respectively. The matrices are defined as:

$$\begin{aligned} Q &= \text{diag}([10, 10, 10, 10]) \\ R &= \text{diag}([10, 10]) \\ Q_f &= 50 \times Q \end{aligned}$$

We enforce the following constraints on the optimization:

- **Boundary Conditions:** The system must begin at the initial state and arrive at the goal state:

$$\mathbf{x}_0 - \mathbf{x}_{start} = \mathbf{0}, \quad \mathbf{x}_N - \mathbf{x}_{goal} = \mathbf{0}$$

- **Actuation Limits:** Control inputs are bounded by the specific torque limits for all k :

$$\mathbf{u}_{min} \leq \mathbf{u}_k \leq \mathbf{u}_{max}$$

- **System Dynamics:** State transitions must satisfy the discretized dynamics using Trapezoidal integration:

$$\mathbf{x}_{k+1} - \mathbf{x}_k - \frac{\Delta t}{2} (f(\mathbf{x}_k, \mathbf{u}_k) + f(\mathbf{x}_{k+1}, \mathbf{u}_{k+1})) = \mathbf{0}$$

So in summary, the trajectory optimization problem can be formulated as:

$$\min_{\mathbf{x}_0, \dots, \mathbf{x}_N, \mathbf{u}_0, \dots, \mathbf{u}_{N-1}} J = \sum_{k=0}^{N-1} (\mathbf{e}_k^T Q \mathbf{e}_k + \mathbf{u}_k^T R \mathbf{u}_k) + \mathbf{e}_N^T Q_f \mathbf{e}_N \quad (4a)$$

$$\text{subject to } \mathbf{x}_0 - \mathbf{x}_{start} = \mathbf{0} \quad (4b)$$

$$\mathbf{x}_N - \mathbf{x}_{goal} = \mathbf{0} \quad (4c)$$

$$\mathbf{x}_{k+1} - \mathbf{x}_k - \frac{\Delta t}{2} (f(\mathbf{x}_k, \mathbf{u}_k) + f(\mathbf{x}_{k+1}, \mathbf{u}_{k+1})) = \mathbf{0}, \quad k = 0, \dots, N-1 \quad (4d)$$

$$\mathbf{u}_{min} \leq \mathbf{u}_k \leq \mathbf{u}_{max}, \quad k = 0, \dots, N \quad (4e)$$

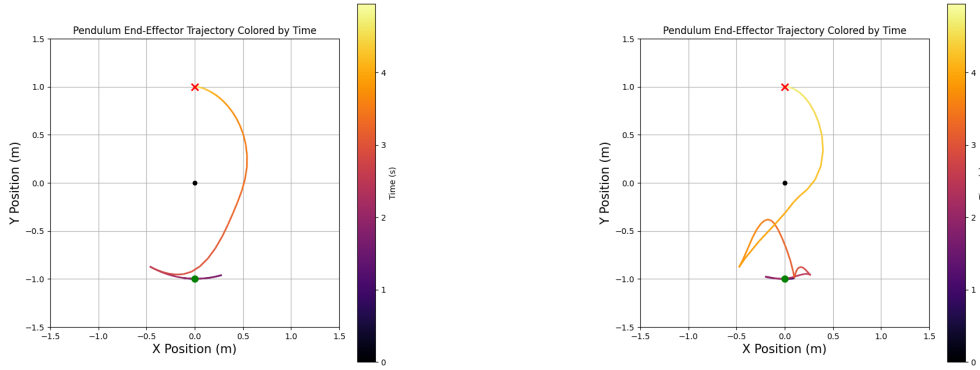
where $\mathbf{x}_{start} = [0, 0, 0, 0]^T$, $\mathbf{x}_{goal} = [\pi, 0, 0, 0]^T$, $\mathbf{u}_{min} = -10 Nm$, and $\mathbf{u}_{max} = 10 Nm$. We can see from the dynamics equality constraint in (4d) that in order to calculate the final state $\mathbf{x}_N$, we need to also include the control input $\mathbf{u}_N$ in the decision vector, even though it is never realized in the system. After formulating the optimization problem, we solve it using the cyipopt solver in Python.

2.1 Results

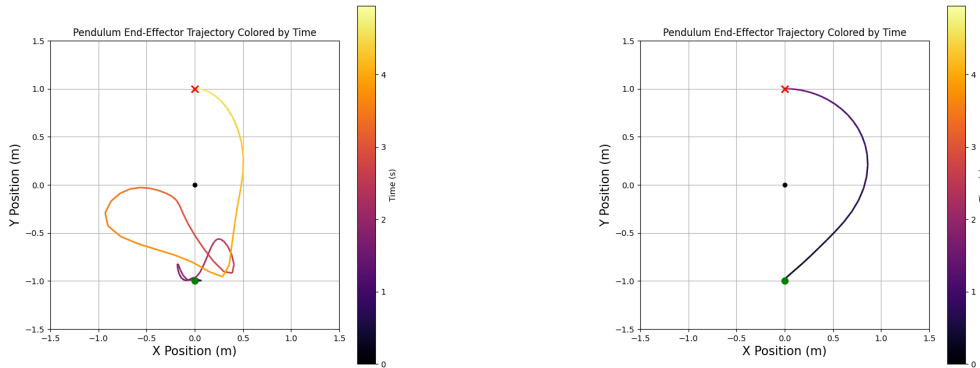
We tested the trajectory optimization framework on the double pendulum system with different Q and R matrices. Specifically, we tested the following configurations:

- **Configuration 1:** $Q = \text{diag}([10, 10, 10, 10])$, $R = \text{diag}([10, 10])$
- **Configuration 2:** $Q = \text{diag}([10, 10, 10, 10])$, $R = \text{diag}([200, 200])$
- **Configuration 3:** $Q = \text{diag}([100, 100, 1, 1])$, $R = \text{diag}([200, 200])$
- **Configuration 4:** $Q = \text{diag}([200, 200, 10, 10])$, $R = \text{diag}([0.1, 0.1])$

The results can be seen in the following plots:

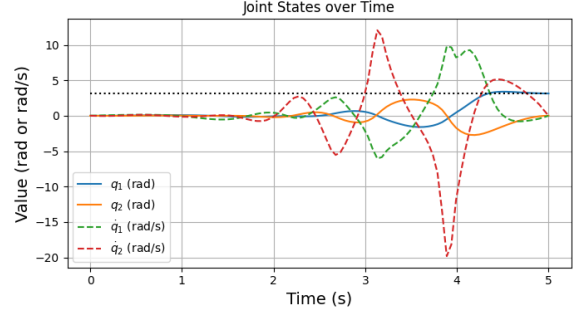
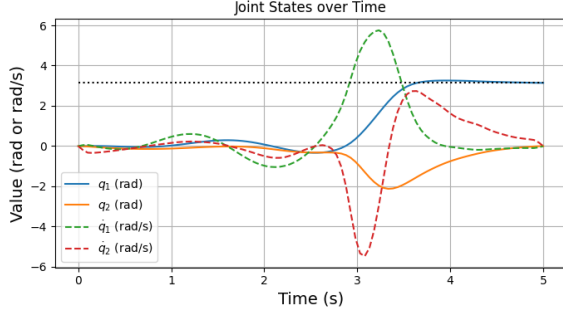


(a) Configuration 1: $Q = \text{diag}([10, 10, 10, 10])$, $R = \text{diag}([10, 10])$ (b) Configuration 2: $Q = \text{diag}([10, 10, 10, 10])$, $R = \text{diag}([200, 200])$

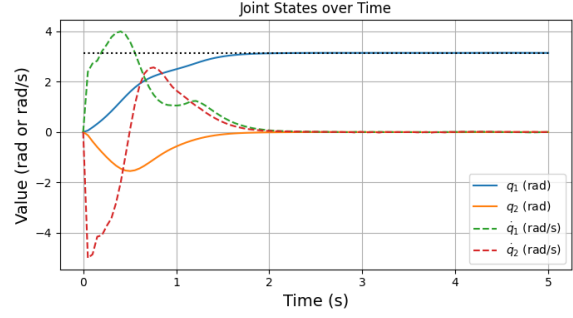
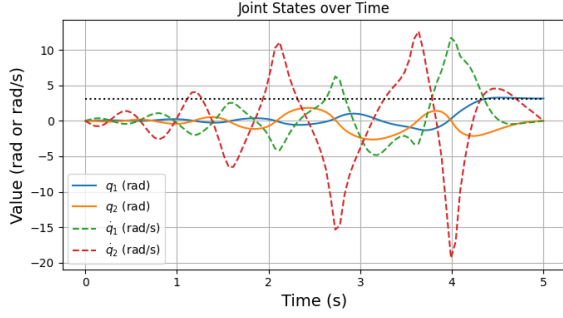


(c) Configuration 3: $Q = \text{diag}([100, 100, 1, 1])$, $R = \text{diag}([200, 200])$ (d) Configuration 4: $Q = \text{diag}([200, 200, 10, 10])$, $R = \text{diag}([0.1, 0.1])$

Figure 1: Optimal Trajectories for Different Configurations of Q and R phase space

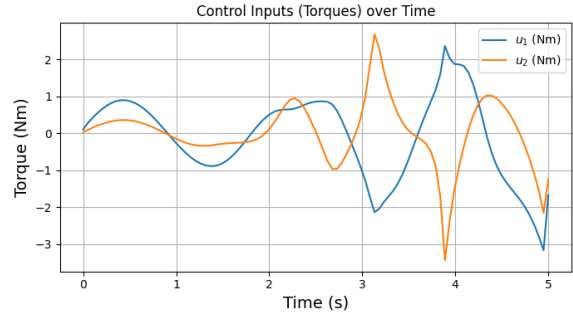
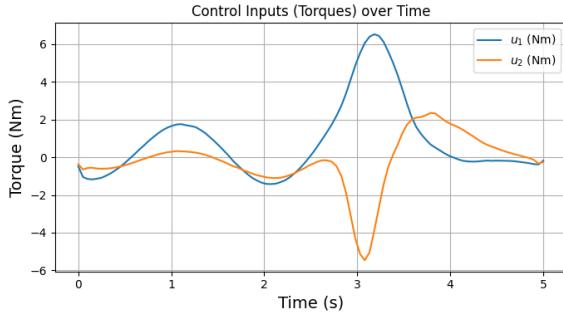


(a) Configuration 1: $Q = \text{diag}([10, 10, 10, 10])$, (b) Configuration 2: $Q = \text{diag}([10, 10, 10, 10])$, $R = \text{diag}([10, 10])$

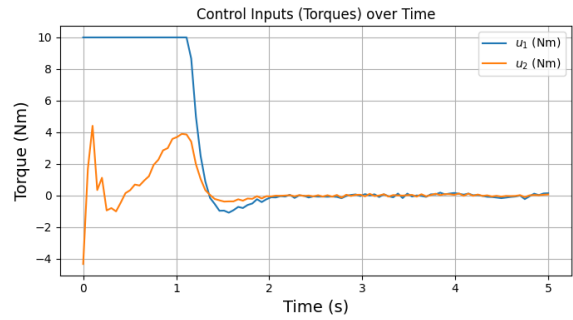
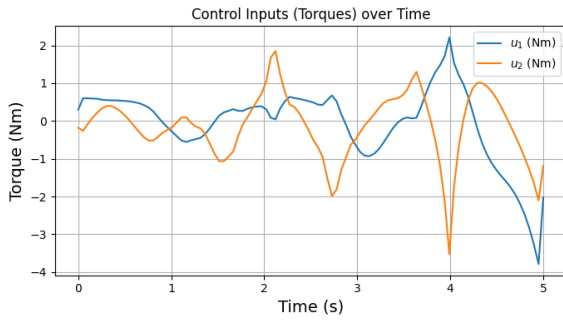


(c) Configuration 3: $Q = \text{diag}([100, 100, 1, 1])$, (d) Configuration 4: $Q = \text{diag}([200, 200, 10, 10])$, $R = \text{diag}([200, 200])$

Figure 2: Optimal Trajectories for Different Configurations of Q and R wrt to time



(a) Configuration 1: $Q = \text{diag}([10, 10, 10, 10])$, (b) Configuration 2: $Q = \text{diag}([10, 10, 10, 10])$, $R = \text{diag}([10, 10])$



(c) Configuration 3: $Q = \text{diag}([100, 100, 1, 1])$, (d) Configuration 4: $Q = \text{diag}([200, 200, 10, 10])$, $R = \text{diag}([200, 200])$

Figure 3: Optimal Trajectories for Different Configurations of Q and R phase space

Based on the 1 we can observe that as we increase the R values, the system tends to utilize the inertia of the pendulums more, resulting in more oscillatory trajectories to reach the goal state. This can be seen by comparing configuration 1 and configuration 2. In configuration 1, with lower R values, the system takes a more direct path to the goal state, while in 2 it exhibits a bouncing around behavior before it generates enough momentum to reach the goal with smaller control inputs.

Additionally, by comparing the first and the second in 2 we see that the more we penalize the control inputs, the longer it takes for the system to reach the goal state for a given Q matrix, while changing the velocity penalties doesn't seem to affect the time significantly.

Finally, in 3 we can see that if we reduce the R values significantly, we can saturate the torque of the first joint which allow us to shoot for more aggressive control inputs, resulting in a very quick and simple trajectory to the goal state.

3 Time Varying LQR Control

After generating the optimal trajectory, we design a time-varying Linear Quadratic Regulator (LQR) to track the trajectory. TVLQR is implemented by linearizing (offline) the system dynamics at each time step along the optimal trajectory and solving the discrete-time Riccati equation backwards in time to compute the optimal feedback gain matrices K_k . The linearized system dynamics around the optimal trajectory $(\mathbf{x}_k^*, \mathbf{u}_k^*)$ are given by:

$$\Delta \mathbf{x}_{k+1} = A_k \Delta \mathbf{x}_k + B_k \Delta \mathbf{u}_k$$

where:

$$A_k = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{(\mathbf{x}_k^*, \mathbf{u}_k^*)}, \quad B_k = \left. \frac{\partial f}{\partial \mathbf{u}} \right|_{(\mathbf{x}_k^*, \mathbf{u}_k^*)}$$

The TVLQR gain matrices K_k are computed (again offline) by solving the discrete-time Riccati equation backwards in time:

$$\begin{aligned} P_N &= Q_f \\ P_k &= Q + A_k^T P_{k+1} A_k - A_k^T P_{k+1} B_k (R + B_k^T P_{k+1} B_k)^{-1} B_k^T P_{k+1} A_k \\ K_k &= (R + B_k^T P_{k+1} B_k)^{-1} B_k^T P_{k+1} A_k \end{aligned}$$

The control law is given by:

$$\mathbf{u}_k = \mathbf{u}_k^* - K_k(\mathbf{x}_k - \mathbf{x}_k^*)$$

The control step is done online during the simulation of the closed-loop system. This makes the TVLQR controller capable of correcting deviations from the optimal trajectory in real-time. Results from the closed-loop simulation with TVLQR tracking can be seen in the following plots:

However, since the TVLQR gains are computed offline based on the linearized dynamics around the optimal trajectory, the controller's performance may degrade if the system deviates significantly from the planned trajectory. To address this, we can retrain a linear regression model to predict the TVLQR gains based on the current state, allowing for a more adaptive control strategy that can better handle deviations from the optimal trajectory.